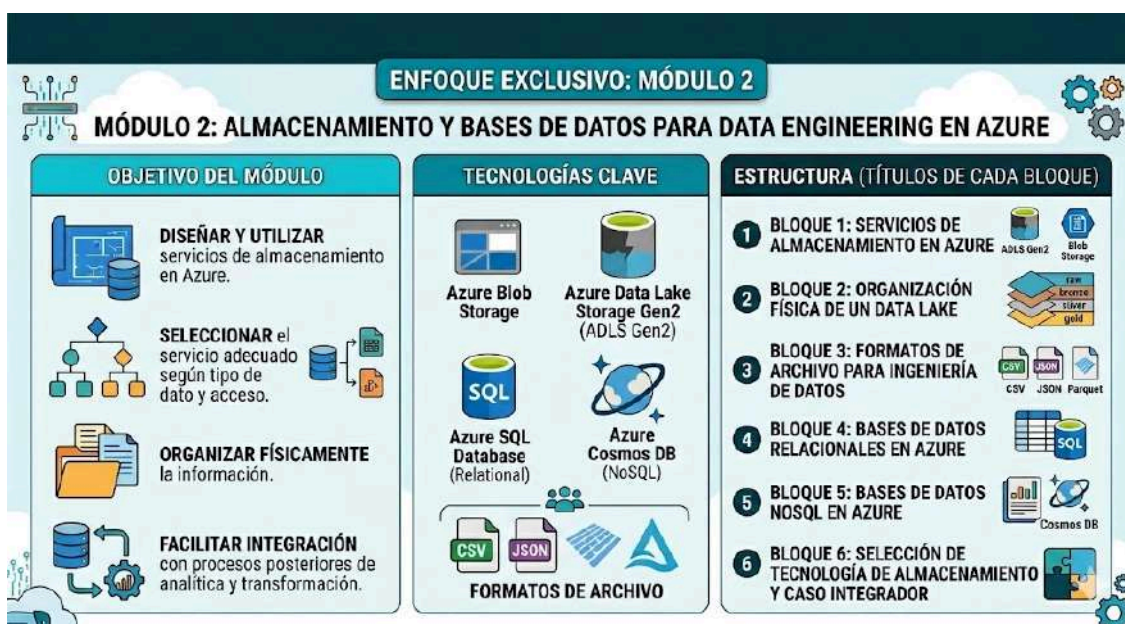




Clase 16 - Soluciones de almacenamiento en Azure

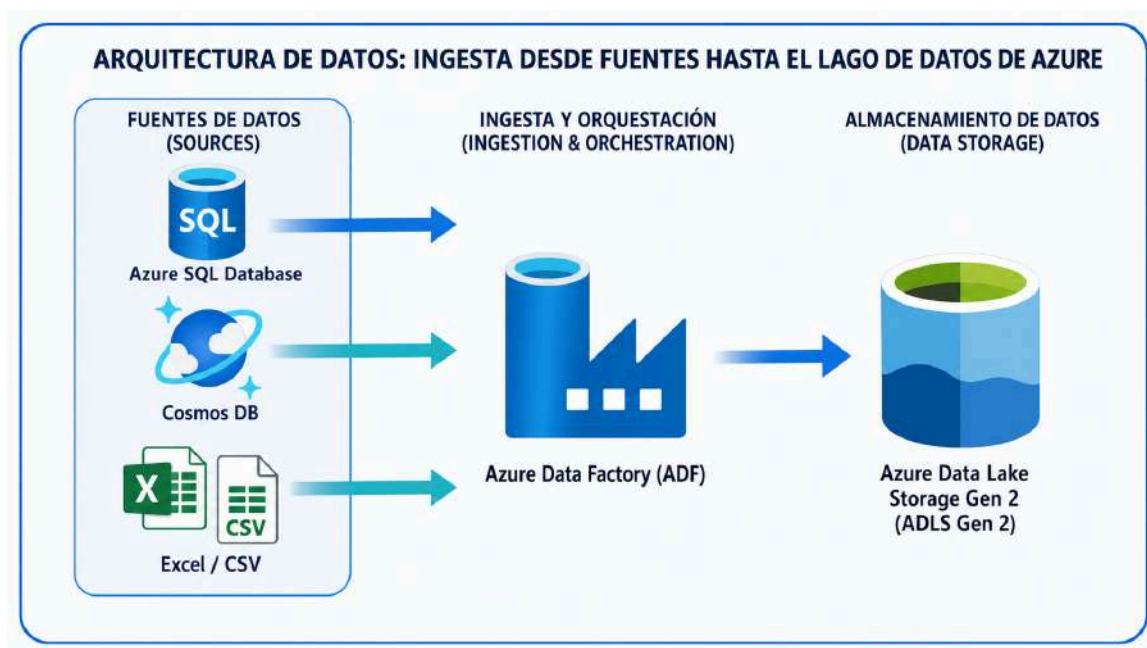
1. Soluciones de almacenamiento y bases de datos en Azure para DE



Cuando trabajamos como Data Engineers, una de las primeras decisiones importantes no es cómo hacer un dashboard, cómo entrenar un modelo de Machine Learning o cómo programar una transformación avanzada. Antes de todo eso, debemos responder una pregunta más básica: **¿Dónde van a vivir los datos?**



Al final de esta sesión se pretende entender en primera aproximación como desarrollar un pequeño pipeline como el que se ilustra:



2. ¿Qué significa almacenar datos en Data Engineering?

En el uso cotidiano, almacenar datos significa simplemente "guardar información". Sin embargo, en Data Engineering el almacenamiento tiene una función mucho más importante.



Por ejemplo, imaginemos una empresa de ventas online. Esta empresa puede tener:



Todos estos datos no tienen la misma estructura ni el mismo uso. Por tanto, no deberían almacenarse necesariamente en el mismo lugar ni de la misma forma.

3. Tipos de datos

Antes de elegir una tecnología, necesitamos entender qué tipo de datos tenemos.

3.1 Datos estructurados

Son datos organizados en filas y columnas. Tienen un esquema claro.

Ejemplo:

cliente_id	nombre	ciudad	fecha_alta
1	Ana Pérez	Madrid	2024-01-10
2	Luis Gómez	Valencia	2024-01-15



3.2 Datos semiestructurados

Son datos que tienen cierta organización, pero no necesariamente una estructura tabular rígida.

Ejemplo en JSON:

```
{
  "evento_id": "evt_001",
  "usuario_id": 45,
  "tipo_evento": "producto_visto",
  "producto_id": 101,
  "fecha_evento": "2024-01-10T10:15:00",
  "dispositivo": "mobile"
}
```

FLEXIBLE
Permite distintos esquemas y campos opcionales.

JERÁRQUICO
Representa datos anidados y relaciones de forma natural.

AUTODESCRIPTIVO
Los datos incluyen su propia estructura (claves y valores).

¿Dónde son comunes los datos semiestructurados?

Este tipo de datos es común en:

- 1 eventos web
- 2 logs
- 3 APIs
- 4 telemetría
- 5 documentos
- 6 catálogos flexibles
- 7 perfiles de usuario

En Azure, una tecnología relevante para este tipo de datos es **Azure Cosmos DB**, especialmente cuando trabajamos con documentos JSON.

Tipos de formatos y su característica principal

• Resumen visual •

Tipo	Característica principal
JSON	clave-valor, jerárquico
XML	etiquetas personalizadas
YAML	legible y simple
HTML	estructura con etiquetas
Logs	texto con patrones
NoSQL docs	esquema flexible

3.3 Datos no estructurados

Son datos que no tienen una estructura tabular o documental clara.



4. Servicios clave de almacenamiento en Azure



5. Azure Storage Account



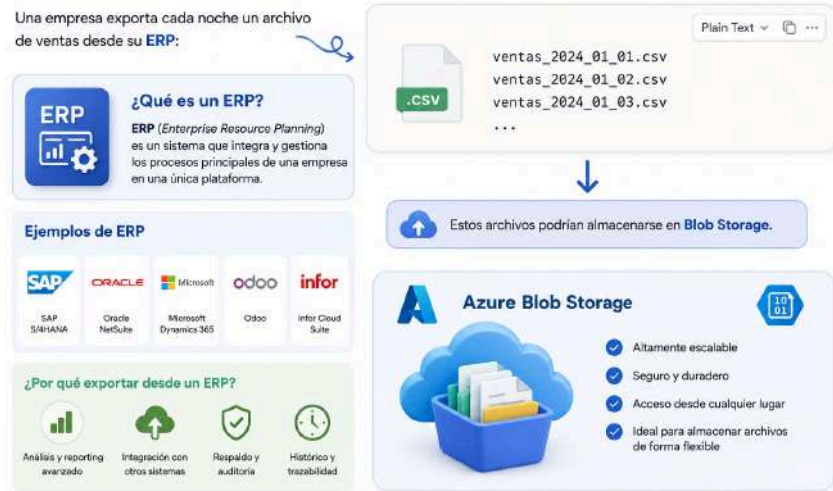
6. Azure Blob Storage



6.1 ¿Cuándo usar Blob Storage?



6.2 Ejemplo



7. Azure Data Lake Storage Gen2



7.1 Diferencia básica entre Blob Storage y ADLS Gen2

7.1 Diferencia básica entre Blob Storage y ADLS Gen2

 Dos servicios de almacenamiento en Azure con propósitos diferentes.

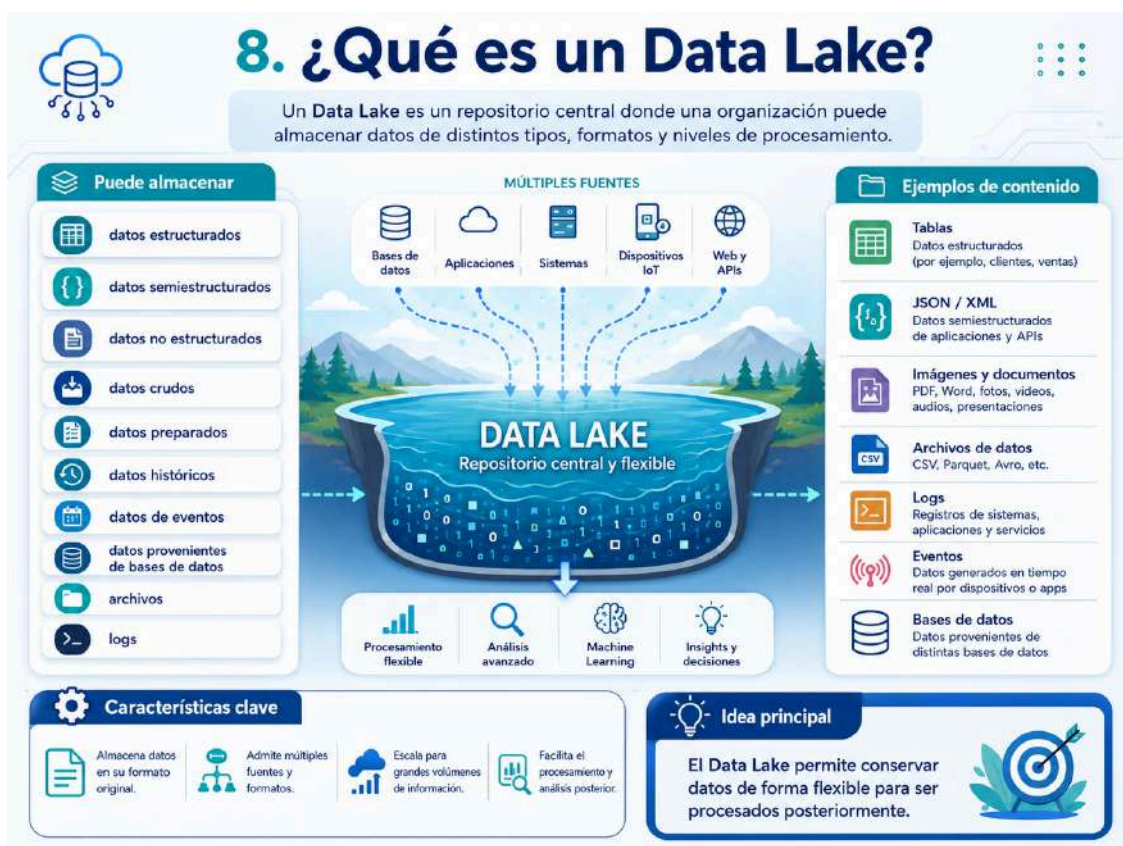
Aspecto	 Azure Blob Storage	 Azure Data Lake Storage Gen2
 Uso principal	Almacenamiento general de objetos	Almacenamiento analítico para datos
 Organización	Contenedores y blobs	Contenedores, directorios y archivos
 Enfoque	Archivos en general	Data Lake y analítica
 Jerarquía de carpetas	Menos orientada a estructura analítica	Espacio de nombres jerárquico
 Uso en Data Engineering	Posible	Muy habitual

 **Blob Storage:** ideal para almacenamiento general

VS

 **ADLS Gen2:** ideal para analítica y Data Engineering

8. ¿Qué es un Data Lake?



8.1 Ejemplo



Actividad 3.1 → Investiga lo siguiente:

Fabric tiene un servicio llamado **Onelake**:

¿ Es un data lake ? ¿ Que features añade OneLake versus un Data Lake ?

¿ Está basado en ADLS Gen2 o no tienen nada que ver?

Establece similitudes y diferencias.

9. Organización física de un Data Lake

9.1 Zona raw

La zona **raw** contiene datos tal como llegan desde la fuente, o con muy poca modificación.



9.2 Zona bronze

La zona **bronze** suele contener datos ya ingeridos y mínimamente estructurados.

9.2 Zona bronze



¿Qué es la zona bronze?

La zona **bronze** es el nivel más básico de un documento. Está pensada para contener datos ya ingeridos y mínimamente estructurados, preparados para su uso y consulta.

Objetivo: ofrecer una base limpia, consistente y simple para los siguientes niveles.

Puede incluir:

1 Corrección básica de nombres

Nombre:
John Doe ✓
John Doe ✓

Estandarización de nombres propios, eliminación de errores tipográficos y formatos inconsistentes.

2 Control inicial de tipos

123 ✓
ABC ✓
[icon] ✓

Asignación de tipos de datos básicos (texto, número, fecha, booleano, etc.).

3 Separación por fecha

[calendar icon]

Organización de los datos en particiones o carpetas por fecha (año/mes/día).

4 Almacenamiento en formato más adecuado

PARQUET

Conversión a formatos de almacenamiento eficientes (ej. Parquet, ORC, CSV optimizado).

5 Normalización mínima

[table icon]

Estandarización mínima de valores (mayúsculas, espacios, nulos, códigos básicos, etc.).

Importante

En este módulo solo introducimos esta zona a nivel conceptual o básico. No haremos transformaciones avanzadas, porque eso corresponde a módulos posteriores.

Diferencias entre la zona RAW y la zona BRONZE

Comparación visual dentro de un flujo de datos

Origen de datos (Sistemas, aplicaciones, dispositivos, archivos, APIs...) → **RAW** (Datos en su estado original) → **BRONZE** (Datos mínimamente preparados)

ZONA RAW

1. Definición	Es la capa de entrada donde se almacenan los datos tal como llegan desde la fuente, con cambios mínimos o sin transformar.
2. Objetivo	Conservar el dato original para trazabilidad, auditoría y reprocesamiento.
3. Características principales	- Datos en estado original o casi original - Puede haber ruido, errores, duplicados o nulos - Se prioriza la fidelidad al origen - Sirve como respaldo del dato fuente
4. Transformaciones permitidas	- Ingesta - Cambio mínimo de nombre o extensión si es necesario - Compresión o partición técnica básica
5. Ejemplo de cómo se ve el dato	csv, json/xml originales, logs crudos, registros IoT sin depurar
6. Almacenamiento y organización típica	Suele organizarse por fuente, fecha de llegada o lote.

ZONA BRONZE

1. Definición	Es la primera capa de preparación donde los datos ya fueron ingeridos y reciben una estructura mínima para facilitar su uso posterior.
2. Objetivo	Disponer de una base simple, consistente y lista para procesos posteriores.
3. Características principales	- Datos mínimamente estructurados - Correcciones básicas de calidad - Tipos de datos iniciales controlados - Mejor formato para almacenamiento y consulta
4. Transformaciones permitidas	- Estandarización básica de nombres - Control inicial de tipos - Separación por fecha - Eliminación de errores obvios - Normalización mínima
5. Ejemplo de cómo se ve el dato	Parquet particionado por fecha, columnas con nombres consistentes, formatos básicos corregidos
6. Almacenamiento y organización típica	Suele organizarse por carpetas/particiones como año/mes/día y formatos más adecuados.

RAW	DIFERENCIAS CLAVE	BRONZE
original	1. Estado del dato	preparado de forma básica
mínimo o nulo	2. Nivel de transformación	básico
sin depurar	3. Calidad del dato	ligeramente mejorada
preservar	4. Propósito	preparar
reproceso y trazabilidad	5. Uso principal	base para siguientes capas

9.3 Zona silver



9.4 Zona gold



10. Convenciones de organización





10.4 Usar nombres claros

Buenas prácticas para nombrar archivos y datasets

Asignar nombres claros y consistentes facilita la búsqueda, comprensión y mantenimiento de los datos.

Un buen nombre debe indicar qué contiene el archivo, su origen o dominio, y, cuando sea útil, la fecha o versión.



Evitar



- 1 archivo1.csv
- 2 datos_final_final.csv
- 3 nueva_tabla.csv



Son nombres ambiguos, poco descriptivos o difíciles de mantener.



Preferir



- 1 ventas_diarias_2024_01_01.csv
- 2 clientes_crm_2024_01_01.json
- 3 eventos_web_2024_01_01.parquet



Indican el contenido, la fuente y la fecha, por lo que son más fáciles de entender y localizar.

¿Qué debe incluir un buen nombre?



1. Contenido: qué datos guarda el archivo



2. Origen o dominio: CRM, ERP, web, marketing, etc.



3. Fecha o periodo: útil para trazabilidad y particionado



4. Formato o extensión: csv, json, parquet



Ejemplo explicado



Recomendación:

usar minúsculas, separadores consistentes (por ejemplo, guion bajo _) y evitar nombres genéricos como final, nuevo o archivo1.

11. Formatos de archivo

No basta con decidir dónde guardar los datos. También debemos decidir **en qué formato** guardarlos.

12.1 CSV

12.1 CSV

CSV significa *Comma-Separated Values*. Es un archivo de texto donde los valores están separados por comas u otro delimitador.

Ejemplo

```
1 cliente_id,nombre,ciudad
2 1,Ana Pérez,Madrid
3 2,Luis Gómez,Valencia
```

Ventajas

- ✓ fácil de leer;
- ✓ fácil de generar;
- ✓ compatible con muchas herramientas;
- ✓ útil para prácticas iniciales.

Limitaciones

- ✗ no conserva tipos de datos de forma estricta;
- ✗ puede tener problemas con comas, acentos o saltos de línea;
- ✗ no es eficiente para grandes volúmenes;
- ✗ no es ideal para analítica avanzada.

12.2 JSON

12.2 JSON

JSON significa *JavaScript Object Notation*. Es muy común en APIs, eventos y bases documentales.

Ejemplo

```
{
  "cliente_id": 1,
  "nombre": "Ana Pérez",
  "ciudad": "Madrid"
}
```

Ventajas

- flexible;
- permite estructuras anidadas;
- muy usado en aplicaciones modernas;
- ideal para eventos y documentos.

Limitaciones

- puede ser más pesado que otros formatos;
- puede ser complejo si hay muchos niveles anidados;
- no siempre es cómodo para análisis tabular.

12.3 Parquet



12.3 Parquet

Parquet es un formato de almacenamiento columnar optimizado para analítica y procesamiento de datos.

A diferencia de **CSV**, que guarda la información fila por fila, **Parquet** organiza los datos por columnas. Esto permite leer solo las columnas necesarias, mejorar el rendimiento de consulta y reducir el espacio de almacenamiento gracias a una compresión más eficiente.

CSV vs Parquet

**CSV**
guarda por filas

ID	Fecha	Producto	Ventas

**Parquet**
guarda por columnas

ID	Fecha	Producto	Ventas

Ventajas

- eficiente para análisis;
- reduce tamaño de almacenamiento;
- conserva tipos de datos;
- permite leer solo columnas necesarias;
- funciona muy bien con Spark, Databricks, Fabric y motores analíticos.

Limitaciones

- no es tan fácil de leer directamente como un CSV;
- requiere herramientas específicas para visualizarlo;
- puede ser menos intuitivo para principiantes.

¿Cuándo usarlo?

Ideal cuando trabajamos con grandes volúmenes de datos, análisis frecuentes y plataformas de datos modernas.

12.4 Delta Lake

12.4 Delta Lake

El almacenamiento transaccional para datos de gran escala.

Delta Lake es una capa de almacenamiento transaccional abierta que añade fiabilidad y rendimiento a los archivos Parquet en tu data lake.

En este módulo solo lo mencionamos de forma introductoria.

Permite capacidades clave:

- Transacciones ACID**
Garantiza datos consistentes incluso con escrituras concurrentes.
- Control de versiones (Time Travel)**
Viaja al pasado y consulta cualquier versión anterior de tus datos.
- Evolución de esquema**
Permite añadir, eliminar o modificar columnas sin romper tus datos.
- Mayor confiabilidad**
Menos corrupción de datos y más integridad en el data lake.

¿Cómo funciona Delta Lake?

Delta Lake añade una capa de transacciones y metadatos sobre archivos Parquet usando un `_delta_log` (transaction log) basado en JSON.

Archivos de datos (Parquet)

- parte-0001.parquet
- parte-0002.parquet
- parte-0003.parquet

Transaction Log (_delta_log)

- { "op": "add", "path": "parte-0001.parquet", "size": 1000000 }
- { "op": "add", "path": "parte-0002.parquet", "size": 1000000 }
- { "op": "add", "path": "parte-0003.parquet", "size": 1000000 }

Tabla Delta (vista lógica)

id	nombre	ciudad	edad
1	Ana	Madrid	28
2	Luis	Valencia	34
3	Marta	Barcelona	31

El `_delta_log` guarda cada cambio (add, remove, update, etc.) y permite reconstruir cualquier versión consistente de la tabla.

1. Transacciones ACID
Operaciones atómicas y consistentes. Si algo falla, los datos no quedan a medias.
Escritura iniciada → Commit exitoso → Rollback (si falla)

2. Control de versiones (Time Travel)
Consulta el estado de tus datos en cualquier momento.
v0 (10:00) → v1 (11:00) → v2 (12:00) → v3 (13:00)
`SELECT * FROM tabla VERSION AS OF 2;`

3. Evolución de esquema
Añade o modifica columnas sin interrumpir tus pipelines.
v1: id, nombre → v2: id, nombre, ciudad

4. Mayor confiabilidad
Detecta corrupciones, valida integridad y asegura datos correctos y completos.
Archivo corrupto → Detectado y aislado

¿Por qué usar Delta Lake?

- Rendimiento**: Optimización para lecturas y escrituras a gran escala.
- Confiabilidad**: Garantiza integridad y consistencia de los datos.
- Flexibilidad**: Ideal para analítica, ML, streaming y BI.
- Ecosistema**: Integrado con Spark, Databricks, Flink y otros motores.

¿Dónde se usa?

- Data Lakes empresariales
- Analítica avanzada
- Machine Learning
- Streaming de datos

Delta Lake se construye sobre archivos Parquet, añadiendo transacciones, esquemas y control de versiones para convertir tu data lake en una fuente de datos confiable y lista para producción.

13. Particionado físico



13. Particionado físico

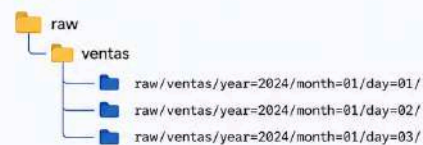
Cómo organizar físicamente los datos en carpetas para mejorar consultas y procesos

El particionado consiste en organizar físicamente los datos en carpetas según un criterio, como fecha, país o región. Esto facilita la gestión, la lectura selectiva y el trabajo incremental.

1. Ejemplos de particionado



Por fecha



✓ Muy útil cuando las cargas llegan cada día y las consultas filtran por periodos.



Por país



✓ Adecuado cuando los análisis suelen separarse por territorio o mercado.

2. ¿Para qué sirve particionar?



1. Mejorar la organización
los datos quedan ordenados de forma lógica.



2. Facilitar búsquedas
localizar información concreta es más rápido.



3. Reducir lectura innecesaria
las consultas leen solo las particiones relevantes.



4. Separar cargas por fecha
ayuda a controlar lotes diarios, mensuales o por periodo.



5. Facilitar procesos incrementales
solo se procesan los datos nuevos o modificados.



3. Error frecuente

Particionar por demasiadas columnas o por campos con muchísimos valores distintos puede crear demasiadas carpetas y complicar la gestión.



Regla práctica: elige criterios que se usen para filtrar consultas y que no generen una explosión de particiones.

No suele ser buena idea particionar por `cliente_id` si hay millones de clientes.



Buena práctica

Escoge particiones útiles para el negocio y para la consulta, como fecha, país o región. Evita claves demasiado granulares.

14. Bases de datos relacionales en Azure

Una base de datos relacional organiza datos en tablas relacionadas entre sí.

En este módulo usamos **Azure SQL Database** como servicio principal de base de datos relacional en Azure.

14.1 ¿Qué es Azure SQL Database?

14.1 ¿Qué es Azure SQL Database?
Base de datos relacional gestionada en Azure

Azure SQL Database es un servicio PaaS de base de datos relacional en la nube de Microsoft Azure, basado en el motor de SQL Server. Permite almacenar, consultar y administrar datos sin tener que encargarse de la infraestructura del servidor, parches o mantenimiento del sistema.

Definición
Azure SQL Database es una base de datos relacional gestionada en Azure, basada en el motor de SQL Server.

¿Con qué podemos trabajar?

- tablas
- columnas
- claves primarias
- claves foráneas
- vistas
- consultas SQL
- relaciones
- integridad referencial

¿Qué aporta Azure SQL Database?

- Servicio administrado en la nube**
Microsoft administra la infraestructura, parches y mantenimiento.
- Alta disponibilidad y escalabilidad**
Escala recursos según la demanda con alta disponibilidad.
- Seguridad y copias de seguridad integradas**
Protección de datos con cifrado, autenticación y backups automáticos.
- Ideal para aplicaciones, reporting y analítica básica**
Rendimiento confiable para aplicaciones y consultas analíticas.

En este módulo también se compara con:

- Azure SQL Database
- SQL Managed Instance
- SQL Server en máquina virtual

Se estudian a nivel comparativo junto con tablas, columnas, tipos de datos, claves, vistas y consultas SQL orientadas a extracción.

Comparativa: Azure SQL Database vs SQL Managed Instance vs SQL Server en VM

Costos, nivel de administración y casos de uso

Azure SQL Database		SQL Managed Instance		SQL Server en máquina virtual	
1) Tipo	• PaaS	1) Tipo	• PaaS	1) Tipo	• IaaS
2) Qué administra Microsoft	• parches, copias de seguridad, alta disponibilidad, actualizaciones, infraestructura	2) Qué administra Microsoft	• parches, copias de seguridad, alta disponibilidad, sistema operativo e infraestructura	2) Qué administra Microsoft	• hardware físico, hipervisor y datacenter
3) Qué administra el cliente	• esquema, usuarios, consultas, rendimiento lógico	3) Qué administra el cliente	• configuración lógica, bases de datos, seguridad, optimización	3) Qué administra el cliente	• SQL Server, sistema operativo, parches, backups, HA/DR, seguridad, configuración
4) Compatibilidad con SQL Server	• alta a nivel de base de datos, pero no compatibilidad completa a nivel de instancia	4) Compatibilidad con SQL Server	• casi 100% compatible con SQL Server; ideal para migración con mínimos cambios	4) Compatibilidad con SQL Server	• 100%: control total de la instancia y del sistema
5) Costos	€ Bajo-Medio Pago por cómputo (DTU o vCore), almacenamiento y backup adicional. Puede ser la opción de entrada más económica; existe opción serverless y elastic pools; reserva y Azure Hybrid Benefit pueden reducir costo.	5) Costos	€€ Medio-Alto Pago por vCores + almacenamiento + backup; la primera parte del almacenamiento base está incluida. Suele costar más que Azure SQL Database, pero menos trabajo operativo que una VM; reservas y Azure Hybrid Benefit pueden reducir costo.	5) Costos	€€-€€€ Variable Pago por la VM + discos + red + licencia SQL (si aplica). Puede aprovechar Azure Hybrid Benefit y reservas; el costo operativo suele ser mayor por la administración.
6) Cuándo usarlo	• aplicaciones cloud-native, web/móvil, microservicios, SaaS, bases individuales	6) Cuándo usarlo	• migrar aplicaciones SQL Server con poca reestructuración; necesidad de características de instancia, varias bases relacionadas	6) Cuándo usarlo	• aplicaciones legadas, dependencias de SO/instancia, control total; funciones no disponibles en PaaS
7) Ventajas	• muy poco mantenimiento • escalado sencillo • alta disponibilidad integrada • bueno para empezar en Azure	7) Ventajas	• gran compatibilidad • menos administración que una VM • ideal para lift-and-shift	7) Ventajas	• máximo control • compatibilidad total • flexibilidad para personalizar
8) Desventajas	• menos control del motor • algunas funciones de instancia no están disponibles	8) Desventajas	• más costoso que SQL Database • no ofrece el mismo control total que una VM	8) Desventajas	• más administración • más responsabilidad operativa • coste total puede crecer rápido

¿Cuál elegir?

Elige Azure SQL Database si buscas simplicidad, menor operación y una base de datos para aplicaciones modernas.

Elige SQL Managed Instance si necesitas migrar SQL Server casi sin cambiar la aplicación.

Elige SQL Server en VM si necesitas control total del sistema y compatibilidad completa.

Resumen rápido de costos

1 Azure SQL Database: normalmente el punto de entrada más económico.

2 Managed Instance: más caro que SQL Database, pero con más compatibilidad.

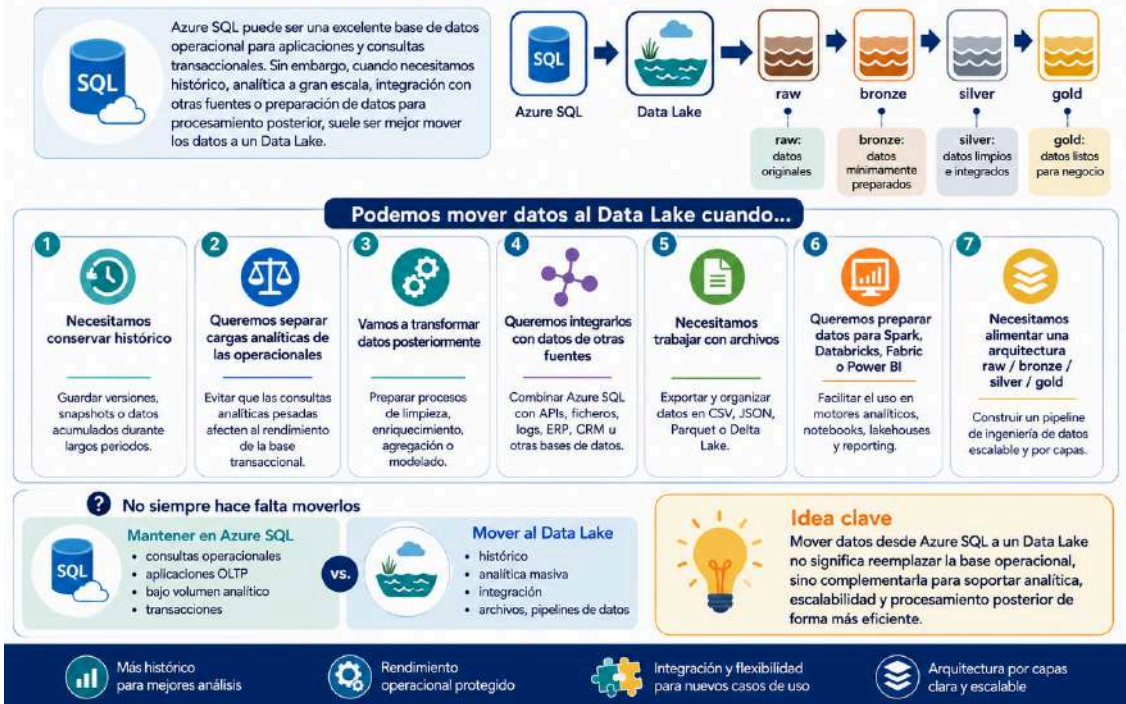
3 SQL Server en VM: coste muy variable; depende mucho del tamaño de la VM, discos y licencias.

Costos orientativos: varían según región, tamaño, licencia, almacenamiento y alta disponibilidad.

17. ¿Cuándo mover datos desde Azure SQL hacia un Data Lake?

17. ¿Cuándo mover datos desde Azure SQL hacia un Data Lake?

Cuándo conviene extraer datos operacionales hacia una arquitectura analítica.



18. Bases de datos NoSQL en Azure

No todas las situaciones encajan bien con una base relacional. A veces los datos son más flexibles, variables o documentales.

En Azure, una opción importante es **Azure Cosmos DB**.

18.1 ¿Qué es una base de datos NoSQL?

18.1 ¿Qué es una base de datos NoSQL?

Modelo de datos flexible para información no relacional

Una base de datos NoSQL es un tipo de base de datos que no depende del esquema relacional clásico de tablas y relaciones rígidas. Está diseñada para manejar grandes volúmenes de datos, estructuras cambiantes y casos donde la **flexibilidad**, la **escalabilidad** y el **rendimiento** son importantes.



¿Cómo puede organizar los datos?



Documentos

Guarda información en estructuras tipo JSON, ideales para aplicaciones modernas.



Clave-valor

Asocia una clave única con un valor, útil para accesos rápidos.



Grafos

Representa nodos y relaciones, ideal para redes, rutas o recomendaciones.



Columnas anchas

Agrupar datos por familias de columnas, útil en analítica y grandes volúmenes.

¿Qué caracteriza a NoSQL?



Esquema flexible



Escalabilidad horizontal



Buen rendimiento para grandes volúmenes



Adaptado a datos semiestructurados y cambiantes



En este módulo nos centramos especialmente en el **modelo documental**, porque es uno de los más utilizados en aplicaciones modernas y servicios cloud.



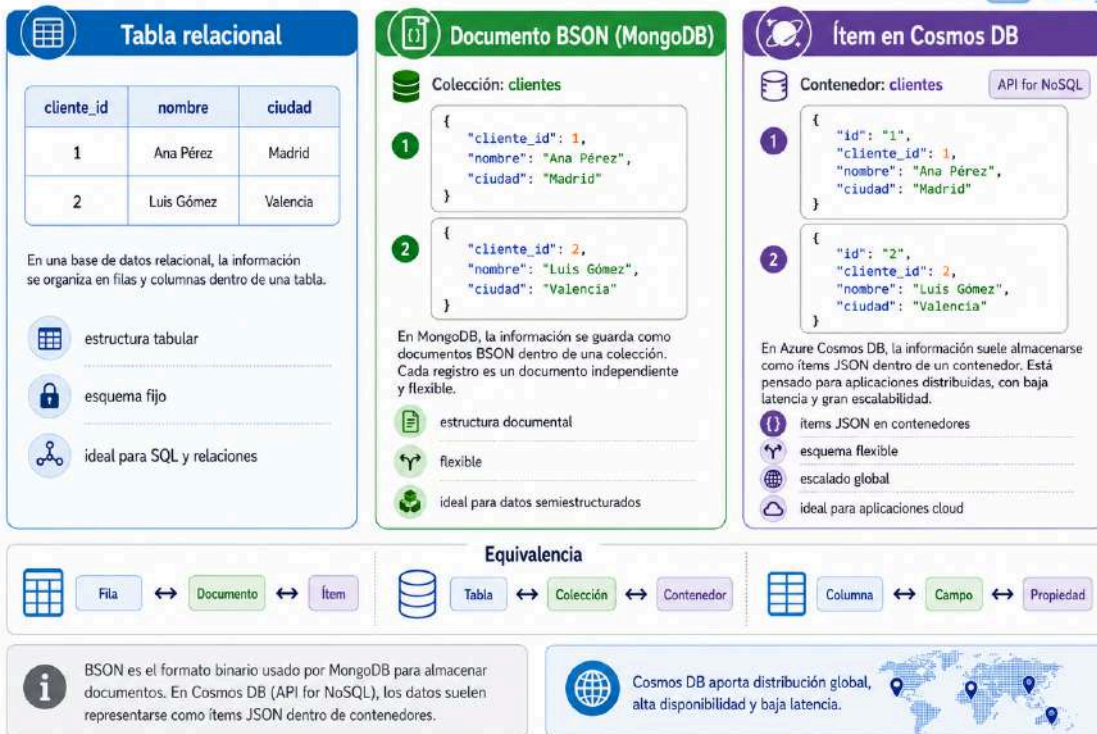
Ejemplo documental

```
{
  "cliente_id": 1,
  "nombre": "Ana Pérez",
  "ciudad": "Madrid"
}
```



Tabla relacional vs documento BSON en MongoDB vs ítem en Cosmos DB

Misma información, distintas formas de representar y almacenar los datos



18.2 Azure Cosmos DB

18.2 Azure Cosmos DB

Azure Cosmos DB es una base de datos **NoSQL** en Azure que permite trabajar con **documentos JSON**.

El módulo incluye los siguientes conceptos clave:

- Bases de datos**: Contenedor lógico para agrupar contenedores.
- Contenedores**: Similar a una tabla o colección donde se almacenan los documentos.
- Documentos JSON**: Datos almacenados en formato JSON, flexibles y semi-estructurados.
- Ítems**: Cada documento JSON es un ítem único dentro de un contenedor.
- Claves de partición**: Clave utilizada para distribuir los datos y garantizar escalabilidad.
- Modelado orientado a consultas**: Diseña los datos pensando en cómo se van a consultar.

18.3 Documento JSON en Cosmos DB

Ejemplo de evento de navegación web:

```
{
  "id": "evt_001",
  "usuario_id": "u123",
  "tipo_evento": "producto_visto",
  "producto_id": "p101",
  "categoria": "informatica",
  "fecha_evento": "2024-01-10T10:15:00",
  "dispositivo": "mobile",
  "pais": "ES"
}
```

¿Por qué usar documentos JSON?

- Esquema flexible**: Cada documento puede tener campos diferentes, sin necesidad de cambiar una estructura fija.
- Agilidad**: Permite almacenar y evolucionar datos rápidamente.
- Natural para aplicaciones modernas**: JSON es el formato nativo de la mayoría de aplicaciones y APIs.
- Escalabilidad automática**: Cosmos DB distribuye los datos automáticamente según la clave de partición y la demanda.

En resumen: Cosmos DB es ideal para escenarios donde los datos cambian, el esquema no es fijo y se requiere **escala, rendimiento y flexibilidad** para aplicaciones modernas.

19. Conceptos básicos de Cosmos DB

19. Conceptos básicos de Cosmos DB

Elementos esenciales para entender cómo se organiza la información en Azure Cosmos DB.

19.1 Base de datos

Es el contenedor lógico principal.

Ejemplo: **retailnova**

Agrupar uno o varios contenedores.

19.2 Contenedor

Agrupar documentos de un tipo o dominio.

Ejemplo: **eventos_web**

Dentro del contenedor se almacenan muchos ítems.

19.3 Ítem

Es cada documento individual.

Ejemplo (JSON):

```
{
  "id": "evt_001",
  "tipo_evento": "producto_visto"
}
```

Cada ítem representa un registro individual.

19.4 Clave de partición

La clave de partición ayuda a distribuir los datos.

Ejemplos de posibles claves:

- /usuario_id
- /pais
- /tipo_evento
- /fecha_evento

★ Elegir una buena clave de partición es importante para rendimiento y escalabilidad.

Permite repartir la carga y evitar concentrar demasiados datos en una sola partición.

JERARQUÍA DE DATOS EN COSMOS DB

Base de datos → Contenedor → Ítem (documento)

retailnova → eventos_web → { "id": "evt_001", "tipo_evento": "producto_visto" }

Distribución de datos

La clave de partición define cómo se distribuyen los datos en las particiones.

Base de datos = agrupación lógica

Contenedor = conjunto de documentos

Ítem = documento individual

Clave de partición = distribución eficiente



20. ¿Cuándo usar Cosmos DB?

Usaremos Azure Cosmos DB en escenarios que requieren flexibilidad de modelo, alto rendimiento y capacidad de escalar fácilmente.



1. Usaremos Cosmos DB cuando...

-  **Trabajamos con documentos JSON.**
Almacena datos en formato JSON de forma nativa.
-  **El esquema puede variar.**
No requiere un esquema fijo ni modificaciones rígidas.
-  **Hay datos semiestructurados.**
Combina flexibilidad de documentos con estructura opcional.
-  **Necesitamos almacenar eventos.**
Guarda eventos con alto volumen y acceso rápido.
-  **Tenemos perfiles de usuario flexibles.**
Cada usuario puede tener atributos distintos.
-  **Recibimos telemetría.**
Ingesta masiva de datos de dispositivos y sensores.
-  **Tenemos catálogos con atributos variables.**
Los productos o ítems pueden tener campos diferentes.
-  **Queremos escalar lecturas y escrituras.**
Escala horizontal automática con baja latencia.



2. Ejemplos comunes

-  **Eventos de navegación web.**
Registra clics, vistas y acciones de los usuarios en tiempo real.
-  **Telemetría de sensores.**
Captura y almacena datos de dispositivos IoT a gran escala.
-  **Perfiles de usuario.**
Almacena información personalizada y dinámica de cada usuario.
-  **Carritos de compra.**
Mantiene el estado del carrito por usuario, en múltiples dispositivos.
-  **Catálogos flexibles.**
Productos o ítems con atributos que pueden cambiar con el tiempo.
-  **Logs de aplicaciones.**
Centraliza y consulta logs de aplicaciones y servicios.



Cosmos DB es ideal cuando se necesita flexibilidad, escalabilidad y baja latencia para datos JSON o semiestructurados.



21. Cosmos DB frente a Azure SQL Database

21. Cosmos DB frente a Azure SQL Database

Comparación visual entre base de datos relacional y base de datos documental / NoSQL



Azure SQL Database

Servicio relacional gestionado en Azure, ideal para datos estructurados y consultas SQL.



Azure Cosmos DB

Base de datos NoSQL en Azure, ideal para documentos JSON, flexibilidad de esquema y gran escalabilidad.

Criterio	Azure SQL Database	Azure Cosmos DB
 Modelo	Relacional	Documental / NoSQL
 Estructura	Tablas	Documentos JSON
 Esquema	Más rígido	Más flexible
 Relaciones	Claves y JOINS	Datos embebidos o referenciados
 Uso típico	Ventas, clientes, pedidos	Eventos, perfiles, telemetría
 Consulta	SQL relacional	Consultas sobre documentos
 Mejor para	Datos estructurados	Datos semiestructurados

 **Idea clave**

- ✓ **Azure SQL Database:** mejor cuando el modelo es estable y relacional.
- ✓ **Azure Cosmos DB:** mejor cuando se necesita flexibilidad, JSON y escalado horizontal.

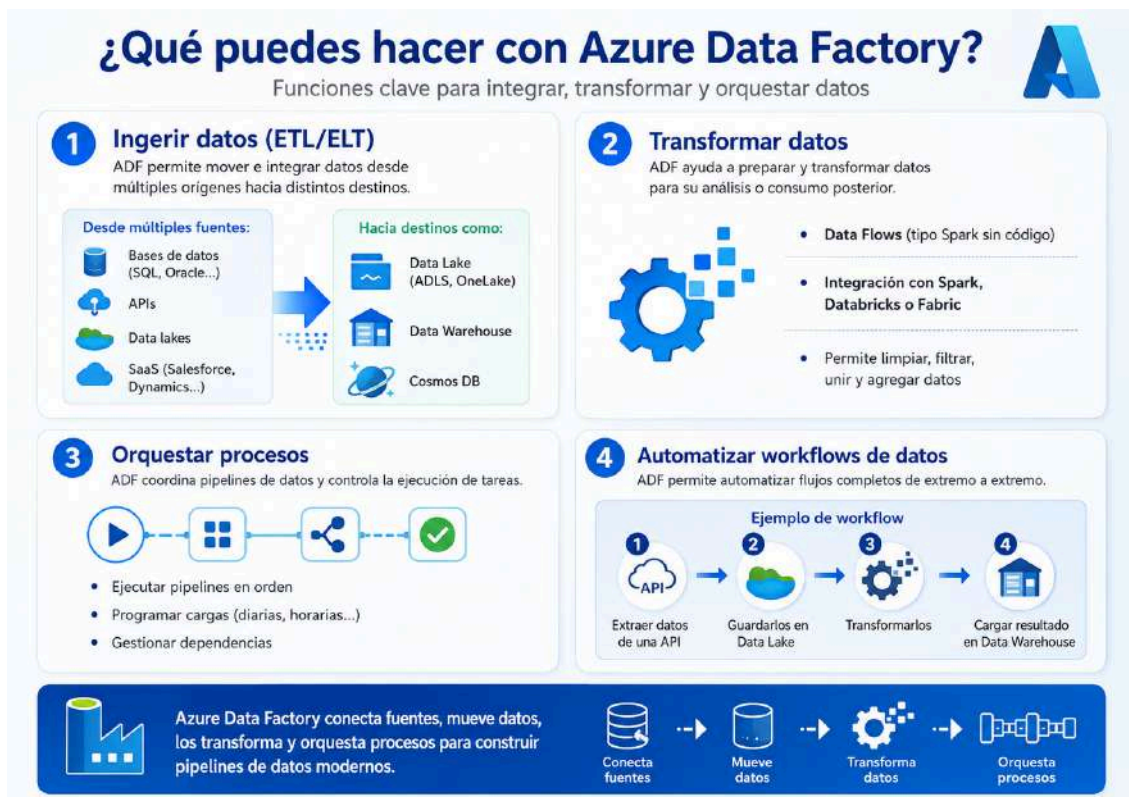
 **Ejemplos rápidos**

-  **Azure SQL Database** → ERP, CRM, ventas, pedidos
-  **Azure Cosmos DB** → eventos web, perfiles de usuario, IoT, catálogos flexibles

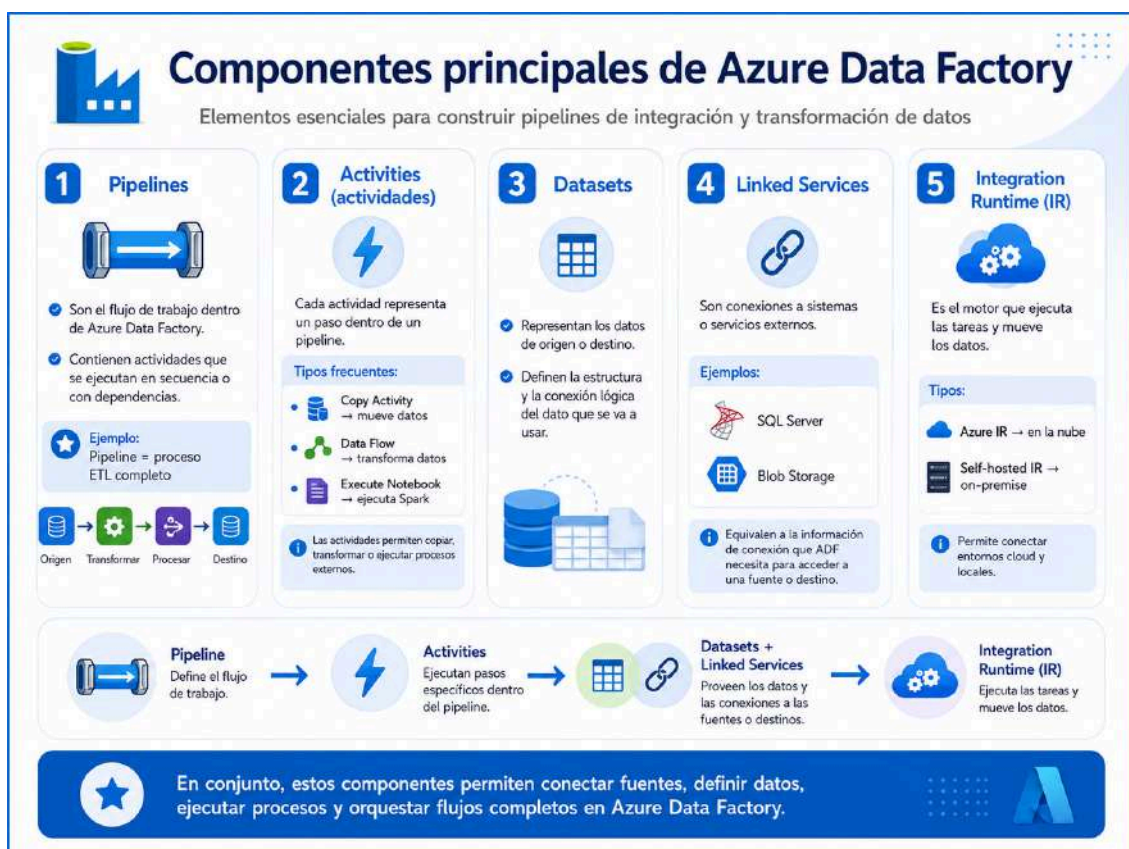
22. Azure Data Factory (ADF)

Azure Data Factory es una herramienta para orquestar flujos de datos entre distintos sistemas y transformarlos para análisis.

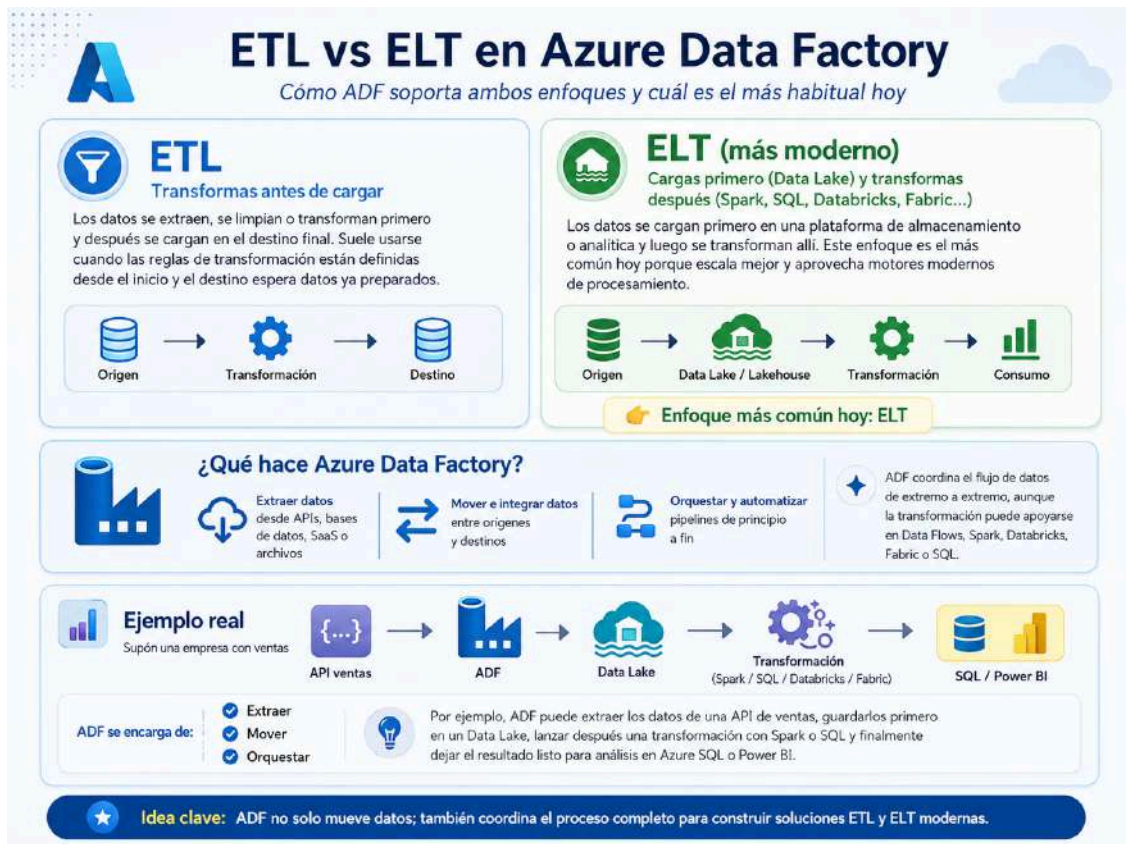
🔧 ¿Qué permite hacer?



Componentes principales



ETL vs ELT en ADF



Actividad 3.2 → Equivalente de ADF en Fabric

Fabric tiene un componente que se llama **Data Factory**. Define y explica que es Data Factory

¿ Es lo mismo que ADF?

Establece similitudes y diferencias y redacta tus conclusiones.

¿ Podría desde Fabric conectarme a datos En Azure/AWS/GCP? De que formas se puede hacer esto y cual sería la utilidad?

¿ Que cosas vamos a practicar en este modulo con ADF?

En las prácticas vamos a usar **Azure Data Factory** como mecanismo mínimo de copia:

```

Azure SQL Database → Azure Data Factory → ADLS Gen2
Cosmos DB → Azure Data Factory → ADLS Gen2
Blob Storage -> ADF -> ADLS Gen2
        
```

Es importante entender el límite: en este módulo **no estudiaremos pipelines completos ni ETL avanzado** porque esos contenidos se reservan para módulos posteriores.

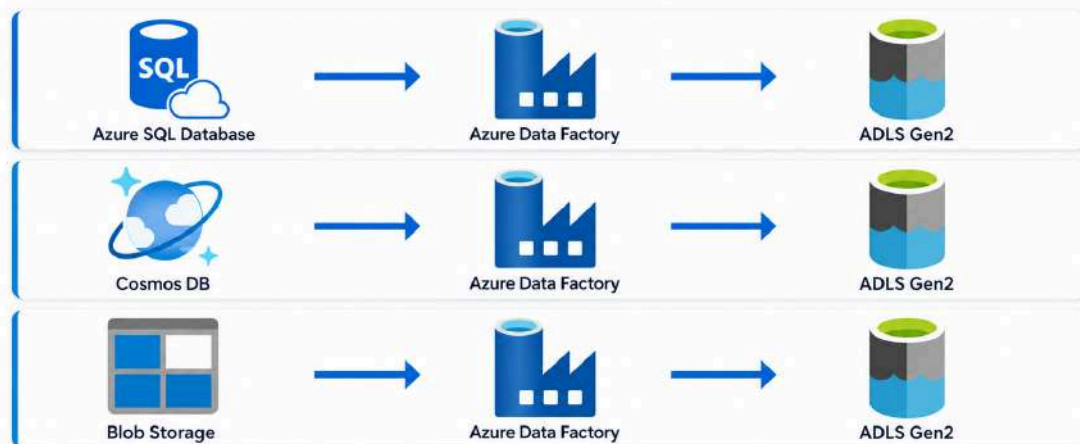
En este módulo lo usaremos únicamente para demostrar una idea básica:

Los datos no solo se almacenan en una fuente; muchas veces deben copiarse hacia un lugar analítico donde puedan ser reutilizados.

ADF nos permite visualizar el movimiento de datos desde una fuente hacia un destino.

Mecanismo mínimo de copia en las prácticas

Usaremos **Azure Data Factory** como servicio de orquestación y copia hacia ADLS Gen2



i ADF actúa como mecanismo mínimo de copia para mover datos desde distintas fuentes hacia ADLS Gen2.

22.2 En este módulo, ADF no se usa para...

En esta parte del curso Azure Data Factory se utiliza solo como mecanismo básico de copia, no como una solución avanzada de orquestación o ingeniería de datos.



orquestación avanzada



transformaciones



triggers complejos



flujos productivos



control de errores avanzado



CI/CD



parametrización compleja



logging avanzado



En este módulo nos centramos solo en copias sencillas de datos para entender el flujo básico con ADF.



Actividad 3.3 Servicios equivalentes en AWS y GCP

En una infografía estilo tabla comparativa coloca los equivalentes de AWS GCP y Fabric de estos servicios de Azure:

- Azure SQL Database
- Azure Data Factory
- Azure Data Lake Storage Gen 2

La tabla debe contener el logo/icono identificativo de cada servicio

Al final de la tabla comparativa escribe un resumen de cada servicio de AWS y GCP que has puesto en la tabla.

Formato de entrega: Word, pdf, powerpoint o markdown (o enlace del repo de Github)